



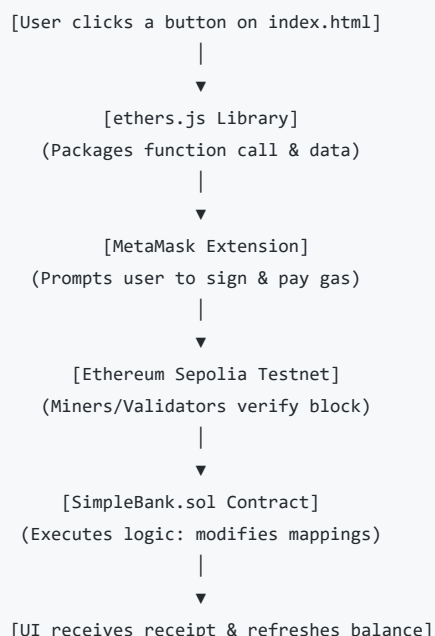
# Simple Banking DApp - Complete Code

## Explanation & Technical Workflow

---

### 1. The Overall Architecture & Workflow

---



### End-to-End Execution Flow

---

1. **User Interaction:** The user enters values (e.g., `0.01` ETH) in `index.html` and clicks a button.
  2. **Data Packaging (ethers.js):** The `ethers.js` client library converts human-readable numbers into blockchain units (**Wei**) and creates a signed transaction payload using the Contract's **ABI**.
  3. **User Signing (MetaMask):** MetaMask detects the transaction request, opens a confirmation dialog, and asks the user to sign and approve the gas fee.
  4. **On-Chain Execution:** The transaction is broadcasted to the **Ethereum Sepolia Testnet**, where validators mine it into a block.
  5. **Contract State Transition:** The EVM (Ethereum Virtual Machine) executes the bytecode of `SimpleBank.sol`, updating its internal state storage ( `mapping` ).
  6. **Receipt & UI Update:** Ethers.js awaits the transaction receipt ( `tx.wait()` ). Once confirmed, the frontend triggers balance getters to update the UI on screen.
-

## 2. Smart Contract Breakdown ( SimpleBank.sol )

---

### 2.1 License & Compiler Version

---

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
```

- `// SPDX-License-Identifier: MIT` : Declares the open-source MIT software license.
- `pragma solidity ^0.8.20` : Specifies that this contract must be compiled using Solidity compiler version `0.8.20` or higher.

### 2.2 State Variables (On-Chain Storage)

---

```
uint256 public constant INITIAL_BALANCE = 100;
uint256 public remainingSupply = 1000000;

mapping(address => uint256) private ethBalances;
mapping(address => uint256) private tokenBalances;
mapping(address => bool) public registered;
address[] public accounts;
```

- `INITIAL_BALANCE` : A constant set to `100` . Every new user who registers is awarded 100 internal Bank Tokens.
- `remainingSupply` : Total supply pool (1,000,000 tokens) reserved for registration bonuses.
- `ethBalances` : A private key-value map ( `address => uint256` ) recording how much native Ether each user has deposited into the bank contract.
- `tokenBalances` : A private key-value map ( `address => uint256` ) storing internal Bank Token bonus balances.
- `registered` : A public key-value map ( `address => bool` ) returning `true` if an address has already registered, otherwise `false` .
- `accounts` : A dynamic array of addresses storing all registered accounts, allowing the frontend to enumerate all users.

### 2.3 Events (Audit Logging)

---

```
event AccountRegistered(address indexed user, uint256 tokenBonus);
event Deposited(address indexed user, uint256 amount);
event Withdrawn(address indexed user, uint256 amount);
event Transferred(address indexed from, address indexed to, uint256 amount);
```

- **Events** write low-cost indexed logs directly into the transaction receipt on the blockchain, allowing indexers (like Etherscan) and frontend applications to track state updates.

---

## 2.4 Modifier (Access Control)

---

```
modifier onlyRegistered() {  
    require(registered[msg.sender], "Account not registered");  
    _;  
}
```

- A reusable validation check. It ensures only addresses that have executed `registernewaccount()` can invoke deposit, withdrawal, or transfer functions.
- `_;` indicates where the body of the modified function will execute once the check passes.

---

## 2.5 Function: `registernewaccount()`

---

```
function registernewaccount() external {  
    require(!registered[msg.sender], "Account already registered");  
    require(remainingSupply >= INITIAL_BALANCE, "Supply exhausted");  
  
    registered[msg.sender] = true;  
    tokenBalances[msg.sender] = INITIAL_BALANCE;  
    remainingSupply -= INITIAL_BALANCE;  
    accounts.push(msg.sender);  
  
    emit AccountRegistered(msg.sender, INITIAL_BALANCE);  
}
```

- **Visibility:** `external` (saves gas by only being callable from outside the contract).
- **Requirements:** Verifies the caller hasn't registered before and that the promotional bonus supply has not run out.
- **Logic:** Sets `registered[msg.sender] = true`, grants 100 tokens, decrements `remainingSupply`, pushes `msg.sender` into the `accounts` array, and emits `AccountRegistered`.

---

## 2.6 Function: `deposit()`

---

```
function deposit() external payable onlyRegistered {
    require(msg.value > 0, "Deposit must be greater than 0");

    ethBalances[msg.sender] += msg.value;

    emit Deposited(msg.sender, msg.value);
}
```

- **payable** : Enables the function to receive native ETH from the caller.
- **msg.value** : The amount of Wei sent alongside the function call.
- **Logic**: Adds **msg.value** to the user's internal **ethBalances** record and emits the **Deposited** event.

---

## 2.7 Function: **withdraw(uint256 amount)**

---

```
function withdraw(uint256 amount) external onlyRegistered {
    require(amount > 0, "Amount must be greater than 0");
    require(ethBalances[msg.sender] >= amount, "Insufficient ETH balance");

    // Update balance BEFORE sending ETH (Prevents Reentrancy Attacks)
    ethBalances[msg.sender] -= amount;

    (bool success, ) = payable(msg.sender).call{value: amount}("");
    require(success, "ETH transfer failed");

    emit Withdrawn(msg.sender, amount);
}
```

- **Security Pattern (Checks-Effects-Interactions)**: The user's internal record **ethBalances[msg.sender]** is deducted **before** the external call transfer is executed to protect against reentrancy vulnerabilities.
- **.call{value: amount}("")** : Transfers the requested Wei directly to the caller's wallet.

---

## 2.8 Function: **transfer(address to, uint256 amount)**

---

```
function transfer(address to, uint256 amount) external onlyRegistered {
    require(to != address(0), "Invalid recipient");
    require(registered[to], "Recipient not registered");
    require(amount > 0, "Amount must be greater than 0");
    require(tokenBalances[msg.sender] >= amount, "Insufficient token balance");

    tokenBalances[msg.sender] -= amount;
    tokenBalances[to] += amount;

    emit Transferred(msg.sender, to, amount);
}
```

- Enforces strict internal transfers: the recipient `to` cannot be the zero address and **must** already be registered in the bank.
- Deducts `amount` from the caller and credits `amount` to the recipient.

---

## 2.9 View Functions (Read-Only)

---

```
function getEthBalance(address user) external view returns (uint256);
function getTokenBalance(address user) external view returns (uint256);
function getAccounts() external view returns (address[] memory);
function contractBalance() external view returns (uint256);
```

- Marked with `view` because they only read storage without modifying it.
- Calls to `view` functions are completely **free of gas fees** when queried by an external client.

---

## 3. Frontend Architecture ( `index.html` )

---

### 3.1 Script Import

---

```
<script src="https://cdn.jsdelivr.net/npm/ethers@6.13.4/dist/ethers.umd.min.js"></script>
```

- Injects **ethers.js v6** into the global browser window scope.

---

### 3.2 Contract Configuration & ABI

---

```
const CONTRACT_ADDRESS = "YOUR_CONTRACT_ADDRESS_HERE";
const CONTRACT_ABI = [
  "function registernewaccount() external",
  "function deposit() external payable",
  "function withdraw(uint256 amount) external",
  "function transfer(address to, uint256 amount) external",
  "function getEthBalance(address user) external view returns (uint256)",
  "function getTokenBalance(address user) external view returns (uint256)",
  "function getAccounts() external view returns (address[])",
  "function contractBalance() external view returns (uint256)",
  "function registered(address) external view returns (bool)"
];
```

- `CONTRACT_ADDRESS` : The deployed location of the contract on the Sepolia network.
- `CONTRACT_ABI` : Human-Readable ABI format supported by ethers.js that defines the functions, parameters, and return types.

---

### 3.3 Automatic Network Switching & Connection ( `connectWallet` )

---

```
async function connectWallet() {
  if (!window.ethereum) {
    alert("Please install MetaMask.");
    return;
  }

  // Force switch to Sepolia (Chain ID 11155111 -> 0xaa36a7)
  try {
    await window.ethereum.request({
      method: 'wallet_switchEthereumChain',
      params: [{ chainId: '0xaa36a7' }],
    });
  } catch (switchError) {
    console.log("Could not switch to Sepolia automatically: ", switchError);
  }

  provider = new ethers.BrowserProvider(window.ethereum);
  await provider.send("eth_requestAccounts", []);
  signer = await provider.getSigner();
  contract = new ethers.Contract(CONTRACT_ADDRESS, CONTRACT_ABI, signer);

  const address = await signer.getAddress();
  document.getElementById("userAddress").innerText = address;
  showStatus("Wallet connected successfully.");

  await checkBalance();
  await checkContractBalance();
}
```

- **Auto-Switch Logic:** Sends the standard JSON-RPC command `wallet_switchEthereumChain` with hex ID `0xaa36a7` to automatically force MetaMask to the Sepolia testnet.
- `provider = new ethers.BrowserProvider(window.ethereum)` : Wraps MetaMask's injected EIP-1193 provider.
- `signer = await provider.getSigner()` : Obtains the user's active signing account.
- `contract = new ethers.Contract(...)` : Instantiates the contract object for calling state-modifying functions.

---

### 3.4 Handling Units ( `parseEther` and `formatEther` )

---

```
// Converting Human ETH to Wei (Deposit)
const tx = await contract.deposit({ value: ethers.parseEther(amount) });

// Converting Human ETH to Wei (Withdrawal)
const tx = await contract.withdraw(ethers.parseEther(amount));

// Converting Wei back to Human ETH for Display
document.getElementById("ethBalance").innerText = ethers.formatEther(eBalance) + " ETH";
```

- Solidity does not support floating point decimals (e.g. `0.01` ).
- `ethers.parseEther(amount)` converts decimal numbers to integer units of **Wei** ( $1 \text{ ETH} = 10^{18} \text{ Wei}$ ).
- `ethers.formatEther(wei)` formats integer Wei back into human-readable strings.

---

### 3.5 Account Switching Event Listener

---

```
if (window.ethereum) {
  window.ethereum.on("accountsChanged", async function () {
    if (!provider) return;
    signer = await provider.getSigner();
    contract = new ethers.Contract(CONTRACT_ADDRESS, CONTRACT_ABI, signer);
    const address = await signer.getAddress();
    document.getElementById("userAddress").innerText = address;
    await checkBalance();
  });
}
```

- Listens to the `accountsChanged` event fired by MetaMask.
- When the user toggles from Account 1 to Account 2, the frontend detects it instantly, re-binds the signer, and refreshes the balances without needing a hard page reload.